Simple Java Design Patterns for Games

From rigid to flexible design with examples

Contents

1 Introduction

A common situation in an introductory object-oriented programming course is the following: a student already knows how to program from previous experience, perhaps in another language or in a more procedural style, and is therefore able to produce a working solution to a problem quite quickly.

That first solution is often perfectly reasonable from the point of view of "making the program work". However, object-oriented programming is not only about writing code that runs. It is also about learning how to structure code so that it remains understandable, maintainable, and open to future extension.

In game development, this becomes especially important. Even a small game tends to grow over time:

- new character types may be added,
- actions may become more varied,
- objects may change behaviour dynamically,

- several systems may need to react to the same event,
- and player actions may need to be configured, stored, or reused.

A design that works well for a tiny prototype may become rigid or difficult to maintain once these extensions appear.

For that reason, learning object-oriented programming requires more than syntax. It requires learning a set of design principles and techniques that help us write code that is easier to extend without constantly rewriting existing parts.

This is exactly the purpose of the examples in this document.

For each situation, we will typically begin with a solution that a student with prior programming experience might naturally write. We will then examine the limitations of that solution and introduce a more object-oriented design that improves flexibility, separation of responsibilities, and extensibility.

The main idea throughout is the following:

Main goal

The objective is not simply to make the code work.
The objective is to organise the code so that new features can be added with less effort and with less risk of breaking what already exists.

2 Scenario 1: Behaviour based on character type

Imagine that we are building a very small turn-based game. At this stage, the game has only two playable character types:

- a `Warrior`, who attacks with a sword,
- and a `Mage`, who attacks by casting fire.

The game engine must execute one turn for whichever character is currently acting.

For now, the objective is only this: given a character, make the game trigger the correct attack.

Because the example is still very small, it is natural to start with a direct and simple solution. Only afterwards will we examine what difficulties appear when the game grows and more character types or actions are added.

2.1 Step 1: No Polymorphism — Rigid Code

A common beginner solution is to let the game engine inspect the concrete type of the object and decide what to do with `if` and `instanceof`.

Example

```
class Warrior {
    public void attackWithSword() {
        System.out.println("Sword slash!");
    }
}

class Mage {
    public void castFire() {
        System.out.println("Fireball!");
    }
}

public class Game {
    public void executeTurn(Object character) {
        if (character instanceof Warrior) {
            ((Warrior) character).attackWithSword();
        } else if (character instanceof Mage) {
            ((Mage) character).castFire();
        }
    }
}
```

Problem

This looks simple, but the design is very rigid.

Design problems

- The `Game` class must know every character type.
- Adding a new class such as `Archer` forces us to modify the game logic (class `Game`).
- The code grows into a long chain of `if/else`.
- This violates the **Open/Closed Principle**.

Why this matters

This is not a good object-oriented solution, even in a tiny example, and in a larger game it quickly becomes fragile. Every extension requires modifying the central logic, and changes to central logic are precisely the ones most likely to introduce bugs with wide-reaching consequences.

2.2 Step 2: Polymorphism with Inheritance

A first improvement is to introduce a common superclass. The game now works with an abstraction instead of concrete classes.

Example

```
1  abstract class Character {
2      abstract void attack();
3  }
4
5  class Warrior extends Character {
6      @Override
7      void attack() {
8          System.out.println("Sword slash!");
9      }
10 }
11
12 class Mage extends Character {
13     @Override
14     void attack() {
15         System.out.println("Fireball!");
16     }
17 }
18
19 public class Game {
20     public void executeTurn(Character c) {
21         c.attack();
22     }
23 }
```

In this example, the abstract class `Character` (lines 1–3) defines the common operation `attack()`, and by declaring it as abstract it forces every subclass to provide its own implementation; as a result, `Warrior` (lines 5–10) and `Mage` (lines 12–17) each define their specific attack behaviour, while `Game` (lines 19–23) can work only with the abstraction `Character` in line 20, without needing to know which concrete subclass it receives.

What improved?

Advantages

- The `Game` class no longer depends on concrete character classes.
- New subclasses can be added without changing `Game`.
- The design uses polymorphism properly.

Remaining limitation

This solution is better, but behavior is still tightly attached to identity. In this model:

- a `Warrior` attacks as a warrior forever,
- a `Mage` attacks as a mage forever.

But games often need more flexibility.

Limitation of inheritance

Inheritance is good at expressing what an object **is**, but it is less good at expressing what an object **does right now**, because the behaviour is normally fixed by its class and therefore does not change at runtime.

For example:

- what if a mage picks up a sword?
- what if a warrior learns a spell?
- what if the player changes weapon during combat?

This leads naturally to the Strategy pattern.

2.3 Step 3: Strategy Pattern

At this point, interfaces start to become especially useful. The reason is that we no longer want the hero to contain one fixed attack implementation inside its own class. Instead, we want the hero to depend only on a common abstraction representing "something that can perform an attack".

The central idea of Strategy is therefore simple: instead of the hero deciding how to attack, it delegates that responsibility to another object. So the hero is no longer the place where the attack algorithm lives.

An interface is a natural tool here because it defines the common contract that all attack behaviours must follow, while allowing different concrete implementations to be plugged in without changing the hero itself.

Example

```
interface AttackStrategy {
    void performAttack();
}

class SwordAttack implements AttackStrategy {
    @Override
    public void performAttack() {
        System.out.println("Physical blade damage!");
    }
}

class MagicAttack implements AttackStrategy {
    @Override
    public void performAttack() {
        System.out.println("Arcane damage!");
    }
}
```

```
18
19 class Hero {
20     private AttackStrategy strategy;
21
22     public Hero(AttackStrategy strategy) {
23         this.strategy = strategy;
24     }
25
26     public void setStrategy(AttackStrategy strategy) {
27         this.strategy = strategy;
28     }
29
30     public void fight() {
31         strategy.performAttack();
32     }
33 }
34
35 public class GamePro {
36     public static void main(String[] args) {
37         Hero h = new Hero(new SwordAttack());
38         h.fight();
39
40         h.setStrategy(new MagicAttack());
41         h.fight();
42     }
43 }
```

What problem does this solve?

Now the same hero can change behavior at runtime.

Main advantage

The attack behavior is no longer glued to the class of the hero. It becomes an interchangeable component.

Advantages

1. Composition over inheritance

With inheritance, behavior tends to be fixed by the class hierarchy. With Strategy, behavior is provided by composition:

- inheritance says: "this object is a sword fighter",
- strategy says: "this object currently uses sword attack behavior".

2. Single Responsibility Principle

The `Hero` class now has a clearer job:

- hold the hero's data,
- manage hero state,
- delegate attack behavior: the actual damage logic belongs to the strategy classes.

3. Easier testing

We can test `MagicAttack` by itself, without creating a full hero object.

Testing benefit

Small, isolated classes are easier to test, debug, and reuse.

Why this may seem more complex at first?

Trade-off

Strategy usually creates more classes. For beginners, this may initially look more complicated than a single class with all the logic inside. But this extra structure pays off as the project grows.

2.4 Why Strategy is Better than Simple Inheritance

Key comparison

Inheritance models identity.
Strategy models interchangeable behavior.

Inheritance is not wrong. It is just not flexible enough when the same object may change how it acts. This is why Strategy is often a better fit for weapons, movement logic, AI decisions, and similar gameplay features.

2.5 Step 4: Decoupling Even More

We can go one step further. The game engine does not even need to know which attack classes exist. Instead, it can simply receive an `AttackStrategy` from somewhere else.

Example

```
public class DecoupledGame {
    public static void main(String[] args) {
        AttackStrategy chosenWeapon = loadFromMenuOrSave();
        Hero h = new Hero(chosenWeapon);

        h.fight();
    }

    private static AttackStrategy loadFromMenuOrSave() {
        return new MagicAttack();
    }
}
```

In this version, the high-level game logic no longer creates the attack behaviour directly inside the `Hero` class. Instead, it first obtains an object of type `AttackStrategy` and then passes it to the hero.

This matters because the code in `main()` only depends on the abstraction `AttackStrategy`. It does not need to know how the attack is implemented internally; it only needs an object that respects that interface.

Design principle

High-level modules should depend on abstractions, not on concrete details.

This example also provides a simple introduction to two important ideas.

First, it illustrates **dependency injection**. The `Hero` needs an attack strategy in order to work, but instead of creating that dependency for itself, it receives it from the outside through the constructor. In other words, the dependency is *injected* into the object.

Second, it illustrates the spirit of **dependency inversion**. The important point is that the higher-level part of the program, such as the game flow, should not be tightly coupled to low-level implementation details such as `MagicAttack` or `SwordAttack`. Instead, both should depend on a common abstraction, here represented by `AttackStrategy`.

Of course, this is still a small example. The method `loadFromMenuOrSave()` still returns a concrete class directly. However, the structure is already better, because it shows that the hero can receive any object implementing `AttackStrategy`, regardless of where that object came from:

- a menu choice,
- a save file,
- a configuration file,

- or some future factory object (preferable if object creation itself becomes complex enough).

So even in this simple version, the code is moving in the right direction: away from hard-coded concrete dependencies and toward programming through abstractions.

3 Scenario 2: Behaviour based on character condition

Suppose now that we no longer want to focus on *which kind of hero* is acting or *which attack style* is being used. Instead, we want to model something different: the same hero should behave differently depending on their current condition during the game. For example, a hero might be:

- healthy,
- wounded,
- poisoned,
- or dead.

The important point is that this is still the *same* hero. What changes is not the identity of the object, but the way it behaves at a given moment. So the new goal is designing the hero so that actions such as moving, attacking, or defending can vary according to the hero's current condition.

As before, we will start with a simple and direct solution, and then examine what problems appear when the number of states and behaviors increases.

3.1 Step 1: Naive State Handling

After Strategy, the next natural pattern is State. The difference is:

- **Strategy**: how an action is performed.
- **State**: how an object behaves depending on its current condition.

A beginner solution is usually an enum plus conditionals.

Example

```
enum State {
    HEALTHY,
    DEAD
}

class Hero {
    State currentState = State.HEALTHY;

    public void move() {
        if (currentState == State.HEALTHY) {
            System.out.println("Running fast!");
        } else if (currentState == State.DEAD) {
            System.out.println("Cannot move, the hero is dead.");
        }
    }
}
```

Problem

This looks harmless with only two states. It becomes messy when we add more:

- wounded,
- poisoned,
- stunned,

- frozen,
- dead.

Why this gets ugly

Every method must be updated whenever a new state is added. That means duplicated logic and many conditionals spread across the class.

3.2 Step 2: The Same Problem Without the State Pattern

Before introducing the State pattern, it is useful to see how the same problem looks like without it. Suppose we want the hero to behave differently depending on whether they are healthy, wounded, or dead. A common beginner solution is to keep one field representing the current condition and then place conditionals inside every method.

Example

```
enum State {
    HEALTHY,
    WOUNDED,
    DEAD
}

class Hero {
    private State currentState = State.HEALTHY;

    public void setState(State newState) {
        this.currentState = newState;
    }

    public void move() {
        if (currentState == State.HEALTHY) {
            System.out.println("Running with agility!");
        } else if (currentState == State.WOUNDED) {
            System.out.println("Limping slowly...");
        } else if (currentState == State.DEAD) {
            System.out.println("Cannot move.");
        }
    }

    public void attack() {
        if (currentState == State.HEALTHY) {
            System.out.println("Strong attack!");
        } else if (currentState == State.WOUNDED) {
            System.out.println("Weak and slow attack.");
        } else if (currentState == State.DEAD) {
            System.out.println("Cannot attack.");
        }
    }

    public void defend() {
        if (currentState == State.HEALTHY) {
            System.out.println("Solid defense!");
        } else if (currentState == State.WOUNDED) {
            System.out.println("Unstable defense...");
        } else if (currentState == State.DEAD) {
            System.out.println("Cannot defend.");
        }
    }
}
```

Why this becomes a problem

The code in Section 3.1 may still seem reasonable, because the example is small and the number of states is limited. However, by Section 3.2 the design is already starting to become more complex in the wrong way.

The problem is that every new state, such as `POISONED`, `STUNNED`, or `FROZEN`, forces us to revisit the same class and expand several methods with additional conditional logic. As a result, the behaviour associated with each state becomes scattered throughout the class instead of being kept in one coherent place.

So the issue is not merely that the code becomes longer. The deeper issue is that it becomes harder to read, harder to maintain, and harder to extend without introducing mistakes.

Why this design scales badly

- Every new state forces us to edit every method.
- The logic for one state is spread across the whole class.
- The class becomes harder to read and maintain.
- This again violates the **Open/Closed Principle**.

This is exactly the moment where the State pattern becomes useful.

3.3 Step 3: State Pattern

Instead of asking in every method *"which state am I in?"*, we move the behavior into separate state objects. The State pattern turns each state into an object. The hero then delegates behavior to the current state.

Example

```
interface CharacterState {
    void onMove();
    void onAttack();
}

class HealthyState implements CharacterState {
    @Override
    public void onMove() {
        System.out.println("Running with agility!");
    }

    @Override
    public void onAttack() {
        System.out.println("Strong attack!");
    }
}

class WoundedState implements CharacterState {
    @Override
    public void onMove() {
        System.out.println("Limping slowly...");
    }

    @Override
    public void onAttack() {
        System.out.println("Weak and slow attack.");
    }
}

class DeadState implements CharacterState {
    @Override
    public void onMove() {
        System.out.println("...");
    }

    @Override
```

```
    public void onAttack() {
        System.out.println("...");
    }
}

class Hero {
    private CharacterState state;

    public Hero() {
        this.state = new HealthyState();
    }

    public void setState(CharacterState newState) {
        this.state = newState;
    }

    public void move() {
        state.onMove();
    }

    public void attack() {
        state.onAttack();
    }
}
```

Advantages

Benefits of State

- Each state keeps its own behavior in one place.
- The `Hero` class becomes simpler.
- New states can be added with less risk of breaking old code.
- State-dependent behavior is easier to understand.

3.4 State Transitions

The strongest part of the State pattern is that the state itself can trigger a transition to another state. For instance, poison may last three turns and then disappear.

Example

```
interface CharacterState {
    void onMove(Hero hero);
    void onAttack(Hero hero);
}

class PoisonedState implements CharacterState {
    private int turnsRemaining = 3;

    @Override
    public void onMove(Hero hero) {
        System.out.println("Poison damage while moving!");
        turnsRemaining--;

        if (turnsRemaining <= 0) {
            hero.setState(new HealthyState());
        }
    }

    @Override
    public void onAttack(Hero hero) {
```

```
        System.out.println("Weak poisoned attack.");
    }
}

class HealthyState implements CharacterState {
    @Override
    public void onMove(Hero hero) {
        System.out.println("Running normally!");
    }

    @Override
    public void onAttack(Hero hero) {
        System.out.println("Normal attack!");
    }
}

class Hero {
    private CharacterState state = new PoisonedState();

    public void setState(CharacterState state) {
        this.state = state;
    }

    public void move() {
        state.onMove(this);
    }

    public void attack() {
        state.onAttack(this);
    }
}
```

This is much cleaner than spreading poison-related conditionals all over the class. The rule is now explicit and local:

- poison causes damage,
- poison lasts a limited number of turns,
- poison eventually ends,
- the hero returns to normal automatically.

4 Strategy vs State

Students often confuse Strategy and State because the class structure looks similar.

Do not confuse these patterns

- **Strategy**: behavior is chosen as an interchangeable algorithm.
- **State**: behavior changes because the object's internal condition changes.

A direct comparison helps students separate the two ideas.

- **Strategy**
  - **Main question:** How is this action performed?
  - **Game example:** sword attack, magic attack, ranged attack.
- **State**
  - **Main question:** How does this object behave in its current condition?
  - **Game example:** healthy, wounded, poisoned, dead.

5 Coupling and Decoupling when combining patters

In object-oriented design, *coupling* refers to how strongly one class or module depends on another. High coupling means that a class knows too many details about others or directly controls them in a rigid way. This makes the system harder to change, test, and extend, because a modification in one place often forces changes in many others.

Decoupling is the opposite: reducing unnecessary dependencies between parts of the code. Decoupled code communicates through abstractions (interfaces, events, messages) rather than concrete implementations. This allows you to add new behavior (new attacks, new character states, new items) without having to modify existing classes all the time.

Both the Strategy and State patterns are tools to help decouple behavior:

- Strategy decouples *what* the hero does (e.g., type of attack) from *how* it is implemented.
- State decouples the hero's behavior *based on internal condition* (healthy, wounded, dead) from the hero class itself.

However, if we combine these patterns carelessly, we can accidentally *reintroduce* strong coupling between them, leading to a design that is hard to change.

5.1 A Badly Coupled Design (Strategy + State Entangled)

In this "bad" design, both the hero and each state know too much about each other and about concrete strategies. We end up with code that is difficult to extend or modify without breaking other parts.

Idea

We want:

- Different `AttackStrategy` implementations (Sword, Magic, Bow, etc.).
- Different `CharacterState` implementations (Healthy, Wounded, Dead, Enraged, etc.).

A naive attempt might be:

1. The `Hero` has a current `CharacterState` and a current `AttackStrategy`.
2. Each state directly picks concrete strategies by itself.
3. The hero sometimes also overrides the strategy, depending on input.

This leads to *duplicated logic* and *hard-wired dependencies* between states and strategies.

Bad Example

```
interface AttackStrategy {
    void performAttack();
}

class SwordAttack implements AttackStrategy {
    @Override
    public void performAttack() {
        System.out.println("Physical blade damage!");
    }
}

class MagicAttack implements AttackStrategy {
    @Override
    public void performAttack() {
        System.out.println("Arcane damage!");
    }
}

interface CharacterState {
    void onMove(Hero hero);
    void onAttack(Hero hero);
```

```
}

class HealthyState implements CharacterState {
    @Override
    public void onMove(Hero hero) {
        System.out.println("Running with agility!");
    }

    @Override
    public void onAttack(Hero hero) {
        // Healthy heroes always use a sword in this design:
        hero.setAttackStrategy(new SwordAttack());
        hero.getAttackStrategy().performAttack();
    }
}

class WoundedState implements CharacterState {
    @Override
    public void onMove(Hero hero) {
        System.out.println("Limping slowly...");
    }

    @Override
    public void onAttack(Hero hero) {
        // Wounded heroes switch to magic (hard-coded rule):
        hero.setAttackStrategy(new MagicAttack());
        hero.getAttackStrategy().performAttack();
    }
}

class DeadState implements CharacterState {
    @Override
    public void onMove(Hero hero) {
        System.out.println("...");
    }

    @Override
    public void onAttack(Hero hero) {
        System.out.println("...");
    }
}

class Hero {
    private CharacterState state;
    private AttackStrategy attackStrategy;

    public Hero() {
        this.state = new HealthyState();
        this.attackStrategy = new SwordAttack();
    }

    public void setState(CharacterState state) {
        this.state = state;
    }

    public void setAttackStrategy(AttackStrategy attackStrategy) {
        this.attackStrategy = attackStrategy;
    }

    public AttackStrategy getAttackStrategy() {
        return attackStrategy;
    }

    public void move() {
        state.onMove(this);
    }
```

```
88
89      public void attack() {
90          // Sometimes the hero overrides the strategy:
91          if (/* some input says use magic */ false) {
92              this.attackStrategy = new MagicAttack();
93          }
94          state.onAttack(this);
95      }
96  }
```

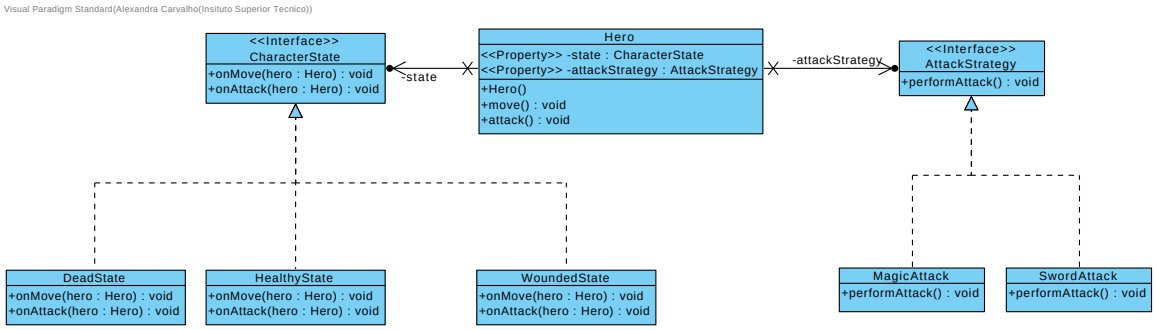

Figure 1: UML of the badly coupled solution. The `CharacterState` methods `onMove(Hero)` and `onAttack(Hero)` receive the `Hero` as a parameter and use that reference to change the hero's concrete attack strategy inside the state classes, creating tight coupling.

Why Is This Badly Coupled?

- **States know concrete strategies.** Each state class creates specific strategy objects (`new SwordAttack()`, `new MagicAttack()`). If you add a new `BowAttack`, you now have to scan all states to see which ones should use it.
- **Hero and states both control the same concern.** The hero sometimes changes the attack strategy based on input, but states *also* change the strategy. The logic for "which attack is active" is spread out and duplicated.
- **Hard-coded rules.** The rule "Healthy = Sword, Wounded = Magic" is baked into the state classes. If game design changes (e.g., Wounded heroes can still use swords but with less damage), you must edit state classes.
- **Difficult to extend.** Adding a new state (e.g., `EnragedState`) means deciding: which strategy should it use? You are tempted to again create concrete strategies inside the new state, growing the coupling graph.

This is a form of tight coupling between:

- States and concrete strategies.
- States and the internal fields of `Hero`.

It reduces the benefits that Strategy and State were supposed to provide.

5.2 A Better Decoupled Design (Clear Responsibilities)

In this improved version, we still combine **State** and **Strategy**, but we keep their responsibilities clearly separated and avoid having them know too much about each other.

- The `Hero` owns:
  - the current `CharacterState`,
  - the current `AttackStrategy`.

- The **Strategy** decides *how* to attack (sword, magic, etc.).
- The **State** decides *whether* and *with what general effect* the hero can perform that attack (e.g. normal, weakened, blocked), but without creating or knowing about concrete strategies.

This keeps the axes of variation orthogonal: you can change the weapon independently of the health state.

Decoupled Example (Without Decorator)

```
interface AttackStrategy {
    void performAttack();
}

class SwordAttack implements AttackStrategy {
    @Override
    public void performAttack() {
        System.out.println("Physical blade damage!");
    }
}

class MagicAttack implements AttackStrategy {
    @Override
    public void performAttack() {
        System.out.println("Arcane damage!");
    }
}

interface CharacterState {
    void onMove();
    void onAttack(AttackStrategy strategy);
}

class HealthyState implements CharacterState {
    @Override
    public void onMove() {
        System.out.println("Running with agility!");
    }

    @Override
    public void onAttack(AttackStrategy strategy) {
        // Healthy heroes perform their current attack normally
        strategy.performAttack();
    }
}

class WoundedState implements CharacterState {
    @Override
    public void onMove() {
        System.out.println("Limping slowly...");
    }

    @Override
    public void onAttack(AttackStrategy strategy) {
        // Wounded heroes can still attack, but it's weaker.
        // We don't change the concrete strategy, just describe the effect.
        System.out.print("With reduced strength: ");
        strategy.performAttack();
    }
}

class DeadState implements CharacterState {
    @Override
    public void onMove() {
        System.out.println("...");
    }

```

```
    @Override
    public void onAttack(AttackStrategy strategy) {
        System.out.println("The hero is dead and cannot attack.");
    }
}

class Hero {
    private CharacterState state;
    private AttackStrategy attackStrategy;

    public Hero(AttackStrategy initialStrategy) {
        this.attackStrategy = initialStrategy;
        this.state = new HealthyState();
    }

    public void setState(CharacterState state) {
        this.state = state;
    }

    public void setAttackStrategy(AttackStrategy attackStrategy) {
        this.attackStrategy = attackStrategy;
    }

    public void move() {
        state.onMove();
    }

    public void attack() {
        state.onAttack(attackStrategy);
    }
}

public class GameDemoSimple {
    public static void main(String[] args) {
        Hero hero = new Hero(new SwordAttack());

        hero.move();
        hero.attack(); // Healthy + Sword

        hero.setState(new WoundedState());
        hero.attack(); // Wounded + (weaker Sword message)

        hero.setAttackStrategy(new MagicAttack());
        hero.attack(); // Wounded + (weaker Magic message)

        hero.setState(new DeadState());
        hero.move();
        hero.attack(); // Dead, cannot attack
    }
}
```

Why is this decoupled?

- `CharacterState` only depends on the `AttackStrategy` *interface*, not on concrete classes like `SwordAttack` or `MagicAttack`.
- Only `Hero` decides *which* attack strategy is equipped.
- States express their effect on attacks (normal, reduced, blocked) without changing the hero's weapon.

This already removes the tight coupling from the previous "bad" example and keeps State and Strategy well separated.

5.3 Extension: Using a Decorator Pattern to Modify Attacks

Building on the decoupled design above, we can go one step further and use the **Decorator** pattern to represent effects like "weakened attack" or "buffed attack" as reusable wrappers around any `AttackStrategy`.

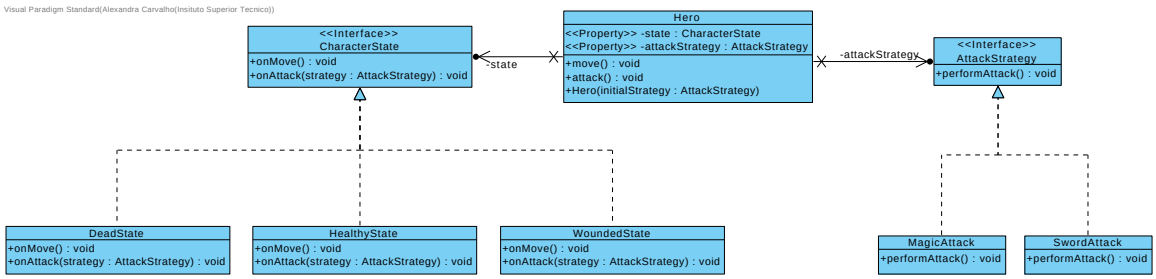

Figure 2: UML of the decoupled solution. The `Hero` owns both the current `CharacterState` and the `AttackStrategy`. The state methods `onMove()` and `onAttack(AttackStrategy)` only work with the `AttackStrategy` interface and simply delegate to it (optionally adding messages), without changing the hero's concrete strategy.

Very informally, a decorator is an object that:

- implements the same interface as the object it decorates, and
- holds a reference to another object of that interface,
- forwarding calls to it while adding some extra behavior.

Here, the idea is the same as before, but instead of just printing "reduced strength", the state can wrap the strategy in another strategy that actually modifies the behavior. In our case, `WeakenedAttackStrategy` implements `AttackStrategy` and internally calls another `AttackStrategy`, but changes its effect (e.g. by printing `"[Weakened]"` first). The state (`WoundedState`) does not care which concrete attack is equipped; it simply wraps "whatever the hero is using now" in a weakened decorator.

Decorator-Based Example

```
interface AttackStrategy {
    void performAttack();
}

class SwordAttack implements AttackStrategy {
    @Override
    public void performAttack() {
        System.out.println("Physical blade damage!");
    }
}

class MagicAttack implements AttackStrategy {
    @Override
    public void performAttack() {
        System.out.println("Arcane damage!");
    }
}

// Decorator: wraps an existing attack strategy and modifies its behavior
class WeakenedAttackStrategy implements AttackStrategy {
    private final AttackStrategy base;

    public WeakenedAttackStrategy(AttackStrategy base) {
        this.base = base;
    }

    @Override
    public void performAttack() {
        System.out.print("[Weakened] ");
        base.performAttack();
    }
}

```

```java
interface CharacterState {
    void onMove();
    void onAttack(AttackStrategy strategy);
}

class HealthyState implements CharacterState {
    @Override
    public void onMove() {
        System.out.println("Running with agility!");
    }

    @Override
    public void onAttack(AttackStrategy strategy) {
        // Healthy: use the base strategy as-is
        strategy.performAttack();
    }
}

class WoundedState implements CharacterState {
    @Override
    public void onMove() {
        System.out.println("Limping slowly...");
    }

    @Override
    public void onAttack(AttackStrategy strategy) {
        // Wounded: wrap the current strategy in a WeakenedAttackStrategy
        AttackStrategy weakened = new WeakenedAttackStrategy(strategy);
        weakened.performAttack();
    }
}

class DeadState implements CharacterState {
    @Override
    public void onMove() {
        System.out.println("...");
    }

    @Override
    public void onAttack(AttackStrategy strategy) {
        System.out.println("The hero is dead and cannot attack.");
    }
}

class Hero {
    private CharacterState state;
    private AttackStrategy attackStrategy;

    public Hero(AttackStrategy initialStrategy) {
        this.attackStrategy = initialStrategy;
        this.state = new HealthyState();
    }

    public void setState(CharacterState state) {
        this.state = state;
    }

    public void setAttackStrategy(AttackStrategy attackStrategy) {
        this.attackStrategy = attackStrategy;
    }

    public void move() {
        state.onMove();
    }

    public void attack() {
```

```
100             state.onAttack(attackStrategy);
101         }
102 }
103
104 public class GameDemoWithDecorator {
105     public static void main(String[] args) {
106         Hero hero = new Hero(new SwordAttack());
107
108         hero.attack(); // Healthy + Sword
109
110         hero.setState(new WoundedState());
111         hero.attack(); // Wounded + [Weakened] Sword
112
113         hero.setAttackStrategy(new MagicAttack());
114         hero.attack(); // Wounded + [Weakened] Magic
115     }
116 }
```

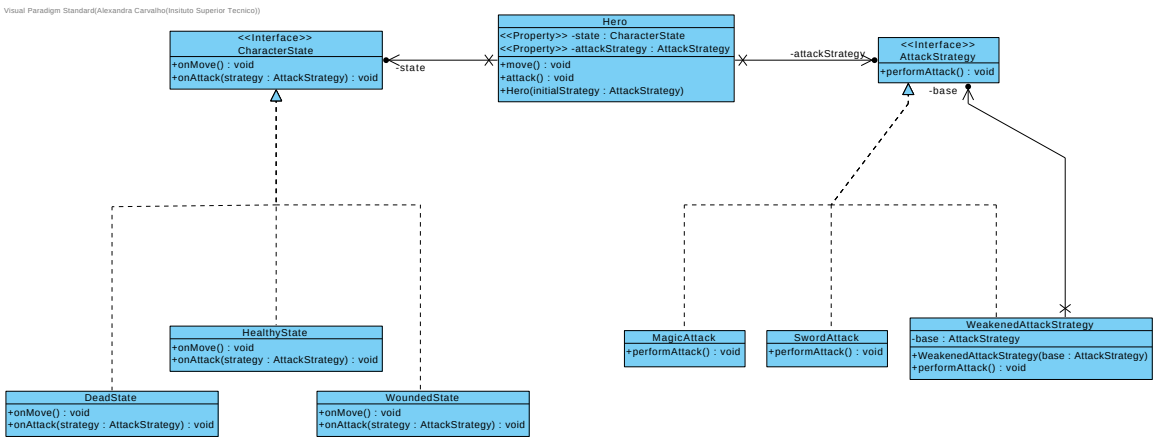

Figure 3: UML of the decoupled solution with decorator. As in the previous UML, the `Hero` owns the current `CharacterState` and `AttackStrategy`, and states receive only the `AttackStrategy` interface. The `WoundedState` illustrates the use of a `WeakenedAttackStrategy` decorator, which wraps the current strategy to modify its behavior without introducing new coupling between states and concrete attack implementations.

How this relates to the previous section?

- The structure (Hero + State + Strategy) is the same as in Section 5.2.
- We only changed `WoundedState.onAttack` to use a decorator object (`WeakenedAttackStrategy`) instead of just printing a message.
- This keeps the *same level of decoupling*, but makes the modification to the attack behavior explicit and reusable.

5.4 Decoupling without Design Patterns: Avoiding Pointer to Pointer Spaghetti

Sometimes students try to "connect everything with everything": objects pass references to other objects, expose internal fields through getters, and change state from many different places. In languages with pointers, this often looks like *pointers to pointers*; in Java, it shows up as long call chains like `a.getB().getC().doX()`.

This creates a form of tight coupling where:

- many classes know each other's internal structure;
- behavior is spread across multiple unrelated places;
- it becomes hard to understand "who is responsible for what".

Bad Example: Changing the Hero Through Many Layers

In this game version, several classes keep references to each other and everyone feels free to change the hero's attack directly.

```
interface AttackStrategy {
    void performAttack();
}

class SwordAttack implements AttackStrategy {
    @Override
    public void performAttack() {
        System.out.println("Physical blade damage!");
    }
}

class MagicAttack implements AttackStrategy {
    @Override
    public void performAttack() {
        System.out.println("Arcane damage!");
    }
}

class Hero {
    AttackStrategy attackStrategy; // package-private for "quick access"

    void attack() {
        attackStrategy.performAttack();
    }
}

// GameWorld holds the hero and exposes it
class GameWorld {
    Hero hero;

    GameWorld(Hero hero) {
        this.hero = hero;
    }

    Hero getHero() {
        return hero; // anyone can now reach the hero directly
    }
}

// Input system changes the hero's strategy by drilling through layers
class InputHandler {
    private final GameWorld world;

    InputHandler(GameWorld world) {
        this.world = world;
    }

    void onKeyPress(int keyCode) {
        // "Pointer to pointer" style access:
        // from InputHandler -> GameWorld -> Hero -> change strategy
        if (keyCode == 1) {
            world.getHero().attackStrategy = new SwordAttack();
        } else if (keyCode == 2) {
            world.getHero().attackStrategy = new MagicAttack();
        }
    }
}

// Some UI element also changes the hero directly
class InventoryUI {
    private final GameWorld world;

```

```
    InventoryUI(GameWorld world) {
        this.world = world;
    }

    void onClickEquipMagicStaff() {
        world.getHero().attackStrategy = new MagicAttack();
    }
}
```

Problems:

- Many different classes (`InputHandler`, `InventoryUI`, etc.) *all* reach into the hero and change its internal field.
- If the representation of attacks inside `Hero` changes, you must update every place that drills down into it.
- There is no single, obvious place where the "equip weapon" rule lives.

This is tight coupling without any pattern: the objects are glued together through chains of references, and behavior is scattered.

Simpler Decoupled Version (No Pattern Needed)

We can improve this just by giving **clear responsibility** to the `Hero` and by avoiding direct access through pointer-like chains. We do *not* need a design pattern for this.

```
interface AttackStrategy {
    void performAttack();
}

class SwordAttack implements AttackStrategy {
    @Override
    public void performAttack() {
        System.out.println("Physical blade damage!");
    }
}

class MagicAttack implements AttackStrategy {
    @Override
    public void performAttack() {
        System.out.println("Arcane damage!");
    }
}

class Hero {
    private AttackStrategy attackStrategy; // now this field is private

    public Hero(AttackStrategy initialStrategy) {
        this.attackStrategy = initialStrategy;
    }

    public void equip(AttackStrategy newStrategy) {
        this.attackStrategy = newStrategy;
    }

    public void attack() {
        attackStrategy.performAttack();
    }
}

class GameWorld {
    private final Hero hero;

    public GameWorld(Hero hero) {
        this.hero = hero;
```

```
    }

    // Hero only expose what others really need:
    public Hero getHero() {
        return hero;
    }
}

class InputHandler {
    private final Hero hero;

    public InputHandler(Hero hero) {
        this.hero = hero;
    }

    public void onKeyPress(int keyCode) {
        if (keyCode == 1) {
            hero.equip(new SwordAttack());
        } else if (keyCode == 2) {
            hero.equip(new MagicAttack());
        }
    }
}

class InventoryUI {
    private final Hero hero;

    public InventoryUI(Hero hero) {
        this.hero = hero;
    }

    public void onClickEquipMagicStaff() {
        hero.equip(new MagicAttack());
    }
}
```

What changed?

- The `Hero` field `attackStrategy` is `private`. Other classes cannot modify it directly.
- Other objects (`InputHandler`, `InventoryUI`) call a clear method: `hero.equip(...)`. The logic of "equipping" is centralized in `Hero`.
- We removed the "pointer to pointer" chain (`world.getHero().attackStrategy = ...`) and instead pass the `Hero` reference directly to the objects that need it.

This is an example of decoupling that does *not* rely on any specific design pattern name. It relies on:

- encapsulation (`private` field),
- clear responsibilities (only `Hero` equips weapons),
- avoiding long chains of calls that expose internal structure.

5.5 Decoupling Between Packages

So far we have mostly talked about coupling between *classes*. The same problem appears one level up, between *packages*. Packages are tightly coupled when:

- they know each other's internal details (implementation classes, fields, etc.);
- there are circular dependencies (`ui` depends on `world` and `world` depends on `ui`);
- a small change in one package forces changes in many others.

In the game context, we can compare a more coupled solution with a more decoupled one, without introducing any new named pattern: just using *encapsulation* and *separation of responsibilities*.

Bad Example: UI Modifying the World Directly

In this version, the UI layer depends directly on the concrete hero class in the `model` package and on its internal fields.

```
game.model  -> Hero, AttackStrategy, MagicAttack, ...
game.ui     -> InventoryUI
```

```java
package game.model;

class Hero {
    AttackStrategy attackStrategy; // not encapsulated

    void attack() {
        attackStrategy.performAttack();
    }
}
```

```java
package game.model;

interface AttackStrategy {
    void performAttack();
}
```

```java
package game.model;

class MagicAttack implements AttackStrategy {
    @Override
    public void performAttack() {
        System.out.println("Arcane damage!");
    }
}
```

```java
package game.ui;

import game.model.Hero;
import game.model.MagicAttack;

public class InventoryUI {
    private final Hero hero; // depends on concrete class in another package

    public InventoryUI(Hero hero) {
        this.hero = hero;
    }

    public void onClickEquipMagicStaff() {
        // UI reaches into the model and changes an internal field
        hero.attackStrategy = new MagicAttack();
    }
}
```

In this design:

- The `ui` package depends directly on the concrete `Hero` class in `model` and on its internal representation (`attackStrategy` field).
- The domain operation "equip a weapon" is not expressed as a clear method in the hero; instead, UI code simulates it by assigning to a field from outside.
- Any change to how `Hero` stores or manages its attacks can break the UI and force changes in the `game.ui` package.

A more decoupled solution will still allow the UI to equip weapons, but will do so through a small, stable public API (for example, a method like `equip(...)`), instead of exposing and modifying internal fields of the hero class across packages.

Decoupling Packages Using a `Hero` Interface

To decouple the UI (or other packages) from the concrete hero implementation, we can introduce a `Hero` interface that exposes only the operations other packages need, such as `equip` and `attack`.

```
game.api    -> Hero, AttackStrategy
game.model  -> HeroImpl, SwordAttack, MagicAttack
game.ui     -> InventoryUI
```

```java
package game.api;

public interface Hero {
    void equip(AttackStrategy strategy);
    void attack();
}
```

```java
package game.api;

public interface AttackStrategy {
    void performAttack();
}
```

```java
package game.model;

import game.api.Hero;
import game.api.AttackStrategy;

public class HeroImpl implements Hero {
    private AttackStrategy attackStrategy; //encapsulated

    public HeroImpl(AttackStrategy initialStrategy) {
        this.attackStrategy = initialStrategy;
    }

    @Override
    public void equip(AttackStrategy newStrategy) {
        this.attackStrategy = newStrategy;
    }

    @Override
    public void attack() {
        attackStrategy.performAttack();
    }
}
```

```java
package game.model;

import game.api.AttackStrategy;

public class MagicAttack implements AttackStrategy {
    @Override
    public void performAttack() {
        System.out.println("Arcane damage!");
    }
}
```

```java
package game.ui;

import game.api.Hero;
import game.model.MagicAttack;

public class InventoryUI {
    private final Hero hero; // depends on the interface, not the impl

```

```
    public InventoryUI(Hero hero) {
        this.hero = hero;
    }

    public void onClickEquipMagicStaff() {
        hero.equip(new MagicAttack());
    }
}
```

Why this helps package decoupling.

- The `ui` package depends on the `Hero` *interface*, not on `HeroImpl` or on its internal fields.
- The concrete implementation `HeroImpl` can change internally without forcing changes in `ui`, as long as the `Hero` interface stays the same.
- The interface exposes exactly what other packages need (e.g. `equip`, `attack`), helping to keep dependencies small and clear.

How to Decide These Interfaces During the Project

In practice, students do not start a project with all interfaces perfectly defined. Instead, interfaces such as `Hero` (with operations like `equip` and `attack`) usually emerge gradually as the design becomes clearer. A few practical guidelines help:

- **Depend on behaviour, not on data.** Instead of exposing fields (`getAttackStrategy()`, `setAttackStrategy(...)`) across packages, expose the *actions* that make sense in the domain: `equip`, `move`, `takeDamage`, etc.
- **Hide implementation details.** When another package wants to do something with the hero, ask: "Can this be expressed as a simple operation on an interface, without exposing how the hero is implemented?" If the answer is yes, that operation is a good candidate for the `Hero` interface.
- **Introduce interfaces when multiple clients appear.** When several parts of the code (UI, input, AI, tests) all need to use the same concept (e.g. the hero), it is a good moment to define an interface in a stable package (e.g. `game.api.Hero`) and let all of them depend on that interface instead of the concrete class.
- **Keep interfaces small and coherent.** Do not put "everything" into one big interface. Group together operations that naturally belong to the same role. For instance, `Hero` might expose only combat-related operations, while inventory management lives in another interface.

In summary, the interfaces used to decouple packages (such as `Hero` and `AttackStrategy`) can be driven by the needs of the clients (UI, input, AI, tests) and by the domain language ("equip", "attack", "move"), rather than by the internal data representation of the classes.

6 Scenario 3 : Reacting to Events in a Decoupled Way

We now want to model a different kind of problem. In many games, when one important event happens, several different parts of the system may need to react. For example, when a hero loses health:

- the health bar should update,
- a warning sound may be played,
- a log or message system may record the event,
- an achievement system might check whether a condition was reached.

The important point is that all these reactions are triggered by the same event, but they belong to different parts of the program. So the new design question is:

> How can one object announce that something happened, without needing to know every system that may want to react to that event?

A direct beginner solution is usually to let one class call every other system manually. That works at first, but it creates strong coupling and makes the code hard to extend. We will therefore begin with that naive solution and then improve it using the Observer pattern.

Problem statement

Suppose that when a hero takes damage, several things should happen:

- the health bar should update,
- a warning sound might play,
- a game log might record the event.

6.1 Step 1: One class directly calls everything

A very common beginner solution is to make the `Hero` class call every other system directly.

```
class HealthBar {
    public void update(int health) {
        System.out.println("UI updated: health = " + health);
    }
}

class SoundSystem {
    public void playLowHealthAlert() {
        System.out.println("Playing low health alert sound!");
    }
}

class GameLog {
    public void record(String message) {
        System.out.println("LOG: " + message);
    }
}

class Hero {
    private int health = 100;

    private HealthBar healthBar = new HealthBar();
    private SoundSystem soundSystem = new SoundSystem();
    private GameLog gameLog = new GameLog();

    public void takeDamage(int amount) {
        health -= amount;

        healthBar.update(health);
        gameLog.record("Hero took " + amount + " damage.");

        if (health < 30) {
            soundSystem.playLowHealthAlert();
        }
    }
}
```

Why is this bad?

Problems with this design

- The `Hero` class knows too much about UI, sound, and logging.
- If we add a new reaction, we must modify `Hero`.
- The gameplay class becomes coupled to presentation and support systems.
- This violates the **Single Responsibility Principle**.

The hero should care about game state, not about every system that reacts to game state.

6.2 Step 2: Observer Pattern

With Observer, the `Hero` only announces that something happened. Other objects choose to listen and react.

Core idea

The subject does not directly control all reactions.
It only notifies interested observers that an event occurred.

Example

```
import java.util.ArrayList;
import java.util.List;

interface HeroObserver {
    void onHealthChanged(int newHealth);
}

class HealthBar implements HeroObserver {
    @Override
    public void onHealthChanged(int newHealth) {
        System.out.println("UI updated: health = " + newHealth);
    }
}

class SoundSystem implements HeroObserver {
    @Override
    public void onHealthChanged(int newHealth) {
        if (newHealth < 30) {
            System.out.println("Playing low health alert sound!");
        }
    }
}

class GameLog implements HeroObserver {
    @Override
    public void onHealthChanged(int newHealth) {
        System.out.println("LOG: hero health changed to " + newHealth);
    }
}

class Hero {
    private int health = 100;
    private List<HeroObserver> observers = new ArrayList<>();

    public void addObserver(HeroObserver observer) {
        observers.add(observer);
    }

    public void removeObserver(HeroObserver observer) {
        observers.remove(observer);
    }

    private void notifyHealthChanged() {
        for (HeroObserver observer : observers) {
            observer.onHealthChanged(health);
        }
    }

    public void takeDamage(int amount) {
        health -= amount;
        notifyHealthChanged();
    }
}
```

```
54
55 public class Game {
56     public static void main(String[] args) {
57         Hero hero = new Hero();
58
59         hero.addObserver(new HealthBar());
60         hero.addObserver(new SoundSystem());
61         hero.addObserver(new GameLog());
62
63         hero.takeDamage(20);
64         hero.takeDamage(60);
65     }
66 }
```

Why is this better?

Now the `Hero` does not know the concrete systems reacting to health changes.

Advantages

- The `Hero` class is less coupled.
- New reactions can be added without changing `Hero`.
- UI, audio, and logging stay separated from gameplay logic.
- The code becomes easier to extend and test.

This is a very natural pattern for games because games are full of events: player died, item collected, quest completed, enemy spotted, and score changed. Indeed, observer is useful whenever one event should trigger several independent reactions.

Why this may make program flow less obvious at first?

Trade-off

Observer reduces coupling, but it can also make the flow of the program less obvious.
A beginner may ask: *"Who reacts when this happens?"*
That is the price of decoupling: the system becomes more flexible, but less direct.

7 Scenario 4: Representing Actions as Objects

Problem statement

We now want to model player actions in a more flexible way.

In a simple game, pressing a key may directly trigger an action such as moving, jumping, or attacking. However, as the game becomes more complex, actions often need to be treated as independent entities.

For example, we may want to:

- change which key triggers which action,
- reuse the same action from keyboard input, menus, or AI,
- store actions in a queue,
- record and replay actions,
- or even support undo in some situations.

So the design problem is the following:

> How can we represent game actions in a way that is independent from the specific place where they are triggered?

A direct beginner solution is usually to place input checks and method calls together inside one controller. This works for a very small example, but quickly becomes rigid when controls, actions, and sources of input grow.

We will therefore start with that straightforward solution and then improve it using the Command pattern.

Problem statement

Suppose we want the player to control a hero with inputs such as:

- press `W` to move up,
- press `S` to move down,
- press `SPACE` to jump.

7.1 Step 1: input handling with direct conditionals

A common beginner solution is to put all input handling directly inside one controller method.

```
class Hero {
    public void moveUp() {
        System.out.println("Hero moves up");
    }

    public void moveDown() {
        System.out.println("Hero moves down");
    }

    public void jump() {
        System.out.println("Hero jumps");
    }
}

class InputHandler {
    private Hero hero;

    public InputHandler(Hero hero) {
        this.hero = hero;
    }

    public void handleInput(String key) {
        if (key.equals("W")) {
            hero.moveUp();
        } else if (key.equals("S")) {
            hero.moveDown();
        } else if (key.equals("SPACE")) {
            hero.jump();
        }
    }
}
```

Why is this bad?

Problems with this design

- The input system is tightly coupled to the hero's concrete methods.
- Adding new actions means editing the input handler.
- Remapping controls becomes awkward.
- Reusing the same action in another place is harder.

This solution is acceptable for a tiny prototype, but it becomes rigid as soon as the control scheme grows.

7.2 Step 2: Command Pattern

With the Command pattern, we represent each action as an object. Instead of asking *"if this key, then call this method"*, we map keys to command objects and execute them.

Core idea

A command object encapsulates a request.
This lets us treat actions as first-class objects.

Example

```
import java.util.HashMap;
import java.util.Map;

interface Command {
    void execute();
}

class Hero {
    public void moveUp() {
        System.out.println("Hero moves up");
    }

    public void moveDown() {
        System.out.println("Hero moves down");
    }

    public void jump() {
        System.out.println("Hero jumps");
    }
}

class MoveUpCommand implements Command {
    private Hero hero;

    public MoveUpCommand(Hero hero) {
        this.hero = hero;
    }

    @Override
    public void execute() {
        hero.moveUp();
    }
}

class MoveDownCommand implements Command {
    private Hero hero;

    public MoveDownCommand(Hero hero) {
        this.hero = hero;
    }

    @Override
    public void execute() {
        hero.moveDown();
    }
}

class JumpCommand implements Command {
    private Hero hero;

    public JumpCommand(Hero hero) {
        this.hero = hero;
    }
```

```
54
55      @Override
56      public void execute() {
57          hero.jump();
58      }
59  }
60
61  class InputHandler {
62      private Map<String, Command> commands = new HashMap<>();
63
64      public void bind(String key, Command command) {
65          commands.put(key, command);
66      }
67
68      public void handleInput(String key) {
69          Command command = commands.get(key);
70          if (command != null) {
71              command.execute();
72          }
73      }
74  }
75
76  public class Game {
77      public static void main(String[] args) {
78          Hero hero = new Hero();
79          InputHandler input = new InputHandler();
80
81          input.bind("W", new MoveUpCommand(hero));
82          input.bind("S", new MoveDownCommand(hero));
83          input.bind("SPACE", new JumpCommand(hero));
84
85          input.handleInput("W");
86          input.handleInput("SPACE");
87      }
88  }
```

Why is this better?

Now input handling is more flexible.

Advantages

- The input handler does not need to know concrete gameplay logic.
- Controls can be remapped easily.
- Commands can be reused from menus, AI, scripts, or network events.
- Actions become easier to queue or log.

A very important game-related advantage

One of the strongest reasons to use the command pattern in games is that commands can be stored. That means we can do things like:

- record a sequence of actions,
- replay actions,
- place actions in a queue,
- implement undo in some genres.

For example, in a turn-based game, each turn could simply store a list of command objects.

Possible drawback

Trade-off

The Command pattern usually creates more classes. For small projects, this may feel verbose compared with directly calling methods.

But for games with configurable controls, multiple actors, or complex actions, the flexibility is often worth it.

8 Final Summary

Final message

Good design is not about making code more complicated.
Good design is about placing behavior in the right place so that the program can grow without collapsing.

Final warning

Bad design may seem acceptable in a very small game. As the project grows, bad design becomes expensive very quickly.

So the real lesson is not simply learn patterns. The real lesson is to understand **why** these patterns appear:

- **Inheritance** reduces some duplication, but can become rigid.
- **Strategy** makes behavior interchangeable.
- **Dependency injection** reduces knowledge of concrete implementations.
- **State** organizes behavior that depends on condition.
- **Observer** decouples events from reactions.
- **Command** turns actions into objects that can be stored and reused.

In other words, the deeper design goals are:

- reduce coupling,
- separate responsibilities,
- keep behavior flexible,
- isolate change,
- make extension easier than modification.